The Engineering of Folding

Numerical Optimization through Origami

Troy Altus

2026-06-01

Table of contents

Chapter 1

Why Fold?

Learning Objectives

- Understand why folding is an engineering strategy, not just an art form
- Identify real deployable structures that use origami-based fold patterns
- Recognize the optimization problem hiding inside every valid crease pattern
- Set up the conceptual vocabulary used throughout the book

There is a medical device called a self-expanding stent that is threaded through an artery in collapsed form, guided to a blockage, and then released — at which point it springs open to its full diameter and holds the vessel wall apart. The collapsed form fits through a catheter 2mm wide. The deployed form spans 30mm. The ratio is not achieved by brute force; it is achieved by geometry. The stent is a folded structure, and the fold pattern determines both how small it gets and how reliably it opens.

This is origami. Not the paper cranes of grade school, but the serious application of fold geometry to problems where the stakes are a patient's aorta, a satellite's power supply, or a telescope lens larger than a building.

1.1 A Brief History of Useful Folding

The engineering use of origami principles has a surprisingly short formal history, given how ancient the art is. The mathematical study of paper folding began in earnest in the 1970s, with David Huffman's 1976 paper on curvature and creases (Huffman 1976) and Koryo Miura's work on deployable space structures. The connection to computational optimization came later still — Lang's 1996 algorithm for automated origami design (Lang 1996) was the first to state the problem explicitly as a constrained program.

What caused the delay? Partly the usual disciplinary walls between mathematics, engineering, and art. Partly the absence of adequate computational tools. But mostly, perhaps, the fact that paper is cheap and the intuition for folding develops faster by hand than by analysis. It took the space program — where paper is unavailable and the cost of a failed deployment is a destroyed satellite — to force the geometry into equations.

The Miura-ori solar array on the ISAS Space Flyer Unit (1995) is the canonical example (Miura

1985). Eight square meters of solar panel, folded into a cylinder 500mm in diameter. The key property: the entire structure has exactly one mechanical degree of freedom. Pull one point and everything moves. Release it and everything locks. The geometry enforces this — it is not an accident of the mechanism, it is a consequence of the fold pattern.

Since then, origami-inspired engineering has appeared in:

- **Aerospace:** solar arrays, telescope sunshields (the James Webb Space Telescope's sunshield uses a variant of the Miura fold), deployable antennae
- **Medicine:** stents, heart valves, surgical instruments, drug delivery capsules
- **Architecture:** folded plate structures, adaptive facades, flat-pack furniture that assembles without fasteners
- **Robotics:** soft robots that change shape by selectively activating folds, grippers, crawlers

1.2 The Optimization Problem Inside Every Fold

Consider the simplest possible origami: a single straight crease across a rectangular sheet. The crease divides the sheet into two panels. Folding along the crease means rotating one panel relative to the other by a fold angle ϕ, measured from the flat state ($\phi = 0$) to fully folded ($\phi = \pi$).

Nothing interesting yet. Now add a second crease, intersecting the first at a vertex. The two fold angles are no longer independent — they are coupled by a constraint: the sheet is made of paper, and paper does not stretch. The angles must satisfy a geometric relationship determined by the vertex geometry.

Add more vertices, and the constraints proliferate. A crease pattern with V interior vertices has $2V$ fold-angle unknowns (two creases at each vertex, roughly) and V geometric constraints plus boundary conditions. The system is generically under-constrained — there are many valid configurations — but some patterns are so highly constrained that only one configuration is possible: the flat state and the folded state, connected by a single continuous path with one degree of freedom.

Finding crease patterns with specific mechanical properties — one DOF, target compaction ratio, target deployed shape — is an optimization problem. The rest of this book works out the details.

1.3 What We Will Build

The book divides into two parts.

Part I covers the geometry. Chapter 2 establishes the mathematical language: rotation matrices at fold lines, the flat-foldability conditions (Kawasaki's theorem, Maekawa's theorem), and how to check whether a given vertex satisfies them. Chapter 3 analyzes the Miura-ori in detail — its parameterization, its single degree of freedom, and why it achieves the compaction ratios that make it useful.

Part II covers the optimization. Chapter 4 treats origami base design as a circle-packing problem and implements Lang's algorithm using `scipy`. Chapter 5 minimizes elastic fold energy

to find equilibrium shapes of partially-folded structures. Chapter 6 solves the inverse problem: given a target deployed geometry, find the crease parameters that achieve it.

Throughout, the emphasis is on the connection between the geometric picture and the numerical algorithm. A fold angle constraint is also a nonlinear equality constraint in a program. A compaction ratio is also an objective function. The vocabulary of origami and the vocabulary of optimization are, in the end, the same vocabulary.

1.4 Summary

- Origami engineering solves real problems: deployable solar panels, self-expanding stents, flat-pack structures.
- Every valid crease pattern is a feasible point in a constrained geometric space.
- Finding crease patterns with desired properties is an optimization problem.
- The rest of this book develops the geometric and numerical tools to solve it.

1.5 Further Reading

The best single-volume introduction to the mathematics of origami is Demaine and O'Rourke (2007). For the engineering applications, Lang (2011) covers origami design from a practitioner's perspective. The original Miura solar array paper (Miura 1985) is short and worth reading in full.

1.6 Exercises

1. **Identify the fold.** Find one example of an origami-inspired engineering application not mentioned in this chapter. Identify what property of the fold makes it useful (compaction ratio, single DOF, specific deployed shape, etc.).

2. **Count degrees of freedom.** A flat sheet with n parallel creases all running the same direction has how many mechanical degrees of freedom? What changes if the creases are not parallel?

3. **The fold angle.** Define a fold angle ϕ for a single crease such that $\phi = 0$ means flat and $\phi = \pi$ means fully folded (the two panels touch). Write an expression for the perpendicular distance between the two far edges of a sheet of width W as a function of ϕ and the crease position x_0.

4. **Constraint counting.** A vertex where four creases meet has four fold angles. How many scalar constraints does flat-foldability impose at that vertex? Is the vertex generically rigid (zero DOF), mobile (one or more DOF), or does it depend on the geometry?

Preface

This book began with a photograph.

It showed a solar panel array folded into a cylinder small enough to fit inside a rocket nosecone, and the caption explained that the entire 8-square-meter panel had been collapsed by a single degree of freedom — one pull on one mechanism caused every crease to fold simultaneously, every panel to stack neatly against its neighbor, with a geometry so beautifully constrained that the only way it *could* fold was the way the designer intended. The origami pattern had been computed. The fold was optimal.

That photograph was taken aboard the ISAS Space Flyer Unit in 1995, and the fold pattern was the Miura-ori, invented by astrophysicist Koryo Miura in the 1970s as a solution to a very specific engineering problem: how do you package a large flat surface inside a small cylinder and deploy it reliably in zero gravity, with no hinges, no motors, and no chance of a second attempt?

Miura's answer — a parallelogram tessellation that collapses along two axes simultaneously — turned out to be one of the most elegant intersections of geometry and engineering in the twentieth century. And it is, at its core, a solution to an optimization problem.

What this book is about

The argument of this book is that origami and numerical optimization are the same subject wearing different clothes. A valid flat-foldable crease pattern is a feasible point in a constrained geometric space. Designing an origami base is a circle-packing problem. Finding the equilibrium shape of a partially-folded structure is an energy minimization. And recovering the crease pattern that produces a desired deployed shape — the inverse problem — is a nonlinear program.

We develop all of these ideas from scratch, with Python code at every step.

Who this book is for

Engineers and scientists who are comfortable with linear algebra and calculus, and who have written some Python but may not have used `scipy.optimize` seriously. No prior origami knowledge is assumed. No prior optimization theory is required beyond the intuition that minimizing a function means finding its lowest point.

Each chapter ends with exercises that range from "verify this by hand" to "implement this extension." The exercises are the book. Reading without working them is like reading a recipe without cooking.

A note on the code

All code runs inside the `pixi` environment defined in `pixi.toml`. To reproduce any result:

```
pixi install
pixi run kernel     # register the Jupyter kernel
pixi run render     # render HTML
pixi run preview    # live preview
```

The only dependencies are `numpy`, `scipy`, and `matplotlib` — all available from conda-forge with native ARM64 builds on Apple Silicon.

Chapter 2

Geometry of a Crease

Learning Objectives

- Represent a fold as a rotation matrix and compute panel positions after folding
- State and apply Kawasaki's theorem for flat-foldability at a vertex
- Apply Maekawa's theorem to count mountain and valley folds
- Write Python functions to check both conditions numerically

A piece of paper has three properties that make it mathematically interesting: it is flat, it is inextensible, and it can be folded. Flatness means the rest configuration is a plane. Inextensibility means distances between points on the surface are preserved during folding — the paper does not stretch. Foldability means that along a crease line, one panel can rotate relative to another by an arbitrary angle.

These three properties together severely constrain what shapes a folded sheet can take. Almost all of the content of this book follows from working out those constraints carefully.

2.1 The Fold Angle

Consider a single straight crease running across a sheet. The crease divides the sheet into two planar panels, which we call the *left panel* and the *right panel* (or panel 1 and panel 2). Define the *fold angle* ϕ as the dihedral angle between the two panels, measured such that:

$$\phi = 0 \quad \text{(flat, unfolded)} \tag{2.1}$$

$$\phi = \pi \quad \text{(fully folded, panels touching)} \tag{2.2}$$

The sign of ϕ distinguishes mountain folds (panel 2 folds *up* relative to panel 1, $\phi > 0$) from valley folds ($\phi < 0$). For a single crease, this sign convention is arbitrary — we can always flip the sheet — but once we have a vertex where multiple creases meet, the relative signs matter.

2.2 Rotation Matrices at Fold Lines

Let the crease lie along the unit vector $\hat{e}$. If panel 1 is fixed in space, then panel 2 is obtained by rotating panel 1 about $\hat{e}$ by angle ϕ. The rotation is given by the Rodrigues formula:

$$R(\hat{e}, \phi) = I\cos\phi + (1 - \cos\phi)\hat{e}\hat{e}^T + \sin\phi\,[\hat{e}]_\times \tag{2.3}$$

where $[\hat{e}]_\times$ is the skew-symmetric cross-product matrix:

$$[\hat{e}]_\times = \begin{pmatrix} 0 & -e_z & e_y \\ e_z & 0 & -e_x \\ -e_y & e_x & 0 \end{pmatrix} \tag{2.4}$$

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

def rotation_matrix(axis, angle):
    """Rodrigues rotation matrix: rotate by angle about axis."""
    e = np.asarray(axis, dtype=float)
    e = e / np.linalg.norm(e)
    c, s = np.cos(angle), np.sin(angle)
    skew = np.array([
        [ 0,    -e[2],  e[1]],
        [ e[2],  0,    -e[0]],
        [-e[1],  e[0],  0   ]
    ])
    return c * np.eye(3) + (1 - c) * np.outer(e, e) + s * skew


def fold_panel(panel_vertices, crease_point, crease_axis, angle):
    """Rotate panel_vertices about a crease line by angle."""
    verts = np.asarray(panel_vertices, dtype=float)
    R = rotation_matrix(crease_axis, angle)
    # Translate so crease_point is origin, rotate, translate back
    return (R @ (verts - crease_point).T).T + crease_point


# Demo: fold a single-crease sheet
fig = plt.figure(figsize=(10, 4))

fold_angles = [0, np.pi / 4, np.pi / 2, 3 * np.pi / 4]
panel1 = np.array([[0, 0, 0], [1, 0, 0], [1, 1, 0], [0, 1, 0]])
panel2_flat = np.array([[0, 0, 0], [-1, 0, 0], [-1, 1, 0], [0, 1, 0]])

for i, phi in enumerate(fold_angles):
```

```
    ax = fig.add_subplot(1, 4, i + 1, projection='3d')
    crease_axis = np.array([0, 1, 0])
    crease_point = np.array([0, 0, 0])
    panel2 = fold_panel(panel2_flat, crease_point, crease_axis, -phi)

    for verts, color in [(panel1, 'steelblue'), (panel2, 'coral')]:
        poly = Poly3DCollection([verts], alpha=0.6, facecolor=color, edgecolor='k', linewid
        ax.add_collection3d(poly)

    ax.set_xlim(-1.2, 1.2)
    ax.set_ylim(-0.2, 1.2)
    ax.set_zlim(-1.2, 0.2)
    ax.set_title(f' = {phi:.2f}', fontsize=9)
    ax.set_axis_off()

plt.suptitle('Single-crease fold at increasing fold angles', y=1.02)
plt.tight_layout()
plt.show()
```

2.3 Vertices and Multi-Crease Constraints

The interesting geometry begins at a *vertex* — a point where two or more creases meet. At a vertex with n creases, there are n fold angles $\phi_1, \ldots, \phi_n$ and n sector angles $\alpha_1, \ldots, \alpha_n$, where α_i is the angle between crease i and crease $i+1$ measured in the flat (unfolded) sheet.

The sector angles satisfy:

$$\sum_{i=1}^{n} \alpha_i = 2\pi \tag{2.5}$$

because they partition the full angle around the vertex.

For the sheet to be foldable (no tearing, no self-intersection at the vertex), the composition of rotations around the vertex must close — the last panel must connect back to the first. This closure condition is the source of all the flat-foldability constraints.

2.4 Kawasaki's Theorem

For a vertex with $2k$ creases (flat-foldable vertices always have an even number of creases), flat-foldability requires:

$$\sum_{i=1}^{k} \alpha_{2i-1} = \sum_{i=1}^{k} \alpha_{2i} = \pi \tag{2.6}$$

In words: the alternating sector angles must each sum to π. For a 4-crease vertex, this means:

$$\alpha_1 + \alpha_3 = \alpha_2 + \alpha_4 = \pi \tag{2.7}$$

This is Kawasaki's theorem (Kawasaki 1989), and it is both necessary and sufficient for flat-foldability (in the local, single-vertex sense).

```
def check_kawasaki(sector_angles):
    """
    Check Kawasaki's theorem for a vertex.
    sector_angles: list of angles in radians, alternating around the vertex.
    Returns (satisfied, odd_sum, even_sum).
    """
    alphas = np.asarray(sector_angles)
    n = len(alphas)
    if n % 2 != 0:
        return False, None, None
    odd_sum  = np.sum(alphas[0::2])   # indices 0, 2, 4, ...
    even_sum = np.sum(alphas[1::2])   # indices 1, 3, 5, ...
    satisfied = np.isclose(odd_sum, np.pi) and np.isclose(even_sum, np.pi)
    return satisfied, odd_sum, even_sum


# Example: standard 4-crease vertex
alphas = [np.pi/3, 2*np.pi/3, np.pi/3, 2*np.pi/3]
ok, s1, s2 = check_kawasaki(alphas)
print(f"Sector angles: {[f'{a:.3f}' for a in alphas]}")
print(f"Kawasaki satisfied: {ok}")
print(f"Odd sum:  {s1:.4f}  (  = {np.pi:.4f})")
print(f"Even sum: {s2:.4f}  (  = {np.pi:.4f})")
```

```
# Visualize: which combinations of ( 1,  2) satisfy Kawasaki for a 4-crease vertex?
# Given  1 +  3 =    →  3 =   - 1
# Given  2 +  4 =    →  4 =   - 2
# Constraint:  1 +  2 +  3 +  4 = 2   →  always satisfied for any  1,  2.
# So for a 4-crease vertex, any ( 1,  2) with  1,  2   (0,  ) works.

alpha1_vals = np.linspace(0.1, np.pi - 0.1, 200)
alpha2_vals = np.linspace(0.1, np.pi - 0.1, 200)
A1, A2 = np.meshgrid(alpha1_vals, alpha2_vals)

# Compaction ratio proxy: total variation in fold space
# For illustration, just show the feasible region
fig, ax = plt.subplots(figsize=(5, 4))
ax.fill_between(alpha1_vals, 0.1, np.pi - 0.1, alpha=0.15, color='steelblue',
                label='Kawasaki feasible region')
ax.set_xlabel('  (rad)')
ax.set_ylabel('  (rad)')
ax.set_title('4-crease vertex: Kawasaki feasible region')
```

```
ax.set_xlim(0, np.pi)
ax.set_ylim(0, np.pi)
ax.axvline(np.pi / 2, color='gray', linestyle='--', linewidth=0.8, label='  = /2')
ax.axhline(np.pi / 2, color='gray', linestyle=':', linewidth=0.8, label='  = /2')
ax.legend(fontsize=8)
plt.tight_layout()
plt.show()
```

2.5 Maekawa's Theorem

Kawasaki tells us about the angles. Maekawa's theorem tells us about the fold directions: at any flat-foldable vertex, the number of mountain folds M and valley folds V must satisfy:

$$|M - V| = 2 \tag{2.8}$$

For a 4-crease vertex, this means either $M = 3, V = 1$ or $M = 1, V = 3$. You cannot have $M = 2, V = 2$ at a flat-foldable vertex.

```
def check_maekawa(fold_types):
    """
    Check Maekawa's theorem.
    fold_types: list of +1 (mountain) or -1 (valley).
    Returns (satisfied, M, V).
    """
    types = np.asarray(fold_types)
    M = np.sum(types == 1)
    V = np.sum(types == -1)
    satisfied = abs(M - V) == 2
    return satisfied, int(M), int(V)


cases = [
    ([1,  1,  1, -1], "M=3, V=1"),
    ([1, -1,  1, -1], "M=2, V=2"),
    ([-1, -1, -1, 1], "M=1, V=3"),
]
for types, label in cases:
    ok, M, V = check_maekawa(types)
    print(f"{label:12s}  →  Maekawa satisfied: {ok}")
```

2.6 A Complete Vertex Checker

```
def is_flat_foldable_vertex(sector_angles, fold_types):
    """
    Check both Kawasaki and Maekawa for a single vertex.
```

```
    sector_angles: alternating sector angles in radians
    fold_types: +1 mountain, -1 valley, one per crease
    """
    kaw_ok, s1, s2 = check_kawasaki(sector_angles)
    mae_ok, M, V   = check_maekawa(fold_types)
    return {
        'kawasaki': kaw_ok,
        'maekawa':  mae_ok,
        'flat_foldable': kaw_ok and mae_ok,
        'odd_angle_sum':  s1,
        'even_angle_sum': s2,
        'mountains': M,
        'valleys':   V,
    }


# Standard Miura-ori vertex
alphas = [np.pi/4, 3*np.pi/4, np.pi/4, 3*np.pi/4]
folds  = [1, -1, 1, -1]   # M, V, M, V
result = is_flat_foldable_vertex(alphas, folds)
for k, v in result.items():
    print(f"  {k}: {v}")
```

2.7 Summary

- A fold is a rotation about a crease line, parameterized by fold angle $\phi \in [0, \pi]$.
- Rodrigues' formula gives the rotation matrix for any axis and angle.
- At a multi-crease vertex, flat-foldability requires Kawasaki's theorem (alternating sector angles sum to π) and Maekawa's theorem ($|M - V| = 2$).
- Both conditions can be checked numerically with simple array operations.

2.8 Further Reading

Demaine and O'Rourke (2007) provides rigorous proofs of Kawasaki's and Maekawa's theorems (Chapters 11–12). Huffman (1976) is the original paper connecting curvature to crease geometry and is surprisingly readable.

2.9 Exercises

1. **Rodrigues by hand.** For a crease along $\hat{e} = \hat{y} = (0, 1, 0)$ and fold angle $\phi = \pi/2$, write out $R(\hat{e}, \phi)$ using Equation 2.3. Apply it to the vector $(1, 0, 0)$ and verify the result makes geometric sense.

2. **Three-crease vertex.** Can a vertex with exactly three creases be flat-foldable? Use Kawasaki's theorem to argue why or why not.

3. **Kawasaki violation.** Write a Python function that, given a set of sector angles that

violate Kawasaki's theorem, returns the *minimum perturbation* to any single sector angle that restores the condition. (Hint: the answer is the difference between the actual alternating sums and π, distributed optimally.)

4. **Maekawa patterns.** For a 6-crease vertex, list all (M, V) combinations that satisfy Maekawa's theorem. How many mountain/valley assignment patterns are there for each valid (M, V) pair?

5. **Closure check.** Implement a function that, given sector angles and fold angles for a 4-crease vertex, multiplies the four rotation matrices in sequence and checks whether the product is the identity. Test it on the Miura-ori vertex from the code block above for $\phi = \pi/4$.

Chapter 3

The Miura-ori

Learning Objectives

- Parameterize the Miura-ori unit cell by sector angle and fold angle
- Derive the compaction ratio as a function of
- Verify that the Miura-ori has exactly one mechanical degree of freedom
- Optimize for a target compaction ratio using `scipy.optimize`

In 1970, Koryo Miura was working on a problem that would have seemed absurd to most engineers: how to fold a sheet of paper so that it could be packed into a compact form and then unfolded with a single, smooth motion — no sequential steps, no user assistance, no stuck panels. The application was solar panels on satellites, where the deployment mechanism is one of the most failure-prone components, and where a failed deployment means a dead spacecraft.

His solution was a crease pattern so simple that it can be drawn with a ruler in five minutes, yet so constrained that the entire structure moves as a single rigid mechanism. It is now called the Miura-ori, and it has become the canonical example of origami engineering.

3.1 The Unit Cell

The Miura-ori is a periodic tessellation. The fundamental unit is a parallelogram-shaped panel, replicated across a rectangular grid and connected at shared creases. The geometry of the unit cell is controlled by two parameters:

- **Sector angle** $\alpha \in (0°, 90°)$: the acute angle between adjacent crease lines at each vertex. When $\alpha = 90°$, the pattern degenerates to a simple accordion fold.
- **Fold angle** $\gamma \in [0°, 90°]$: the angle between the panel and the horizontal, measuring how far the structure has been folded. $\gamma = 0°$ is fully folded (collapsed); $\gamma = 90°$ is flat (deployed).

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Polygon
from matplotlib.collections import PatchCollection
```

```
def miura_unit_cell(alpha, gamma, a=1.0, b=1.0):
    """
    Compute the 3D vertex positions of one Miura-ori unit cell.
    alpha: sector angle (radians)
    gamma: fold angle (radians), 0=collapsed, pi/2=flat
    a, b: panel dimensions
    Returns: dict of vertex positions
    """
    # From Schenk & Guest (2013): exact kinematic model
    # Panel half-dimensions
    sa, ca = np.sin(alpha), np.cos(alpha)
    sg, cg = np.sin(gamma), np.cos(gamma)

    # In-plane projection of crease lengths
    l  = a * np.sqrt(1 - (ca * sg)**2)   # projected crease length

    # Vertex 0 at origin
    V = {}
    V[0] = np.array([0.0, 0.0, 0.0])
    V[1] = np.array([a,   0.0, 0.0])

    # Fold direction vector
    d = np.array([ca * sg, sa, ca * cg]) / np.sqrt(1 - (ca * sg)**2 + (ca * cg)**2 - 1 + 1)
    # Simplified: use the closed-form from Wei et al. (2013)
    denom = np.sqrt(sa**2 + (ca * cg)**2)
    V[2] = V[1] + b * np.array([-ca * sg, sa * cg / denom * np.sqrt(sa**2 + (ca*cg)**2),
                                sa * sg / denom * np.sqrt(sa**2 + (ca*cg)**2)])
    V[3] = V[0] + (V[2] - V[1])
    return V


def miura_dimensions(alpha, gamma, m=4, n=4):
    """
    Compute overall dimensions of m×n Miura-ori tessellation.
    Returns (width, height, thickness).
    """
    sa, ca = np.sin(alpha), np.cos(alpha)
    sg, cg = np.sin(gamma), np.cos(gamma)

    # Closed-form from Schenk & Guest (2013), Eq. 5-7
    # (normalized by panel dimension a=b=1)
    W = 2 * m * np.sqrt(1 - (ca * sg)**2)          # width along x
    L = n * 2 * sa * cg / np.sqrt(sa**2 + (ca*cg)**2) if gamma > 0 else 2 * n * sa  # length al
    T = 2 * ca * sg * sa / np.sqrt(sa**2 + (ca*cg)**2) if gamma > 0 else 0  # thickness
    return W, L, T
```

```
# Plot: deployed dimensions vs fold angle for several  values
gammas = np.linspace(0.01, np.pi / 2, 200)
alphas_deg = [30, 45, 60, 75]

fig, axes = plt.subplots(1, 3, figsize=(12, 4))
labels = ['Width W', 'Length L', 'Thickness T']

for alpha_deg in alphas_deg:
    alpha = np.radians(alpha_deg)
    dims = np.array([miura_dimensions(alpha, g) for g in gammas])
    for j in range(3):
        axes[j].plot(np.degrees(gammas), dims[:, j], label=f' ={alpha_deg}°')

for j, ax in enumerate(axes):
    ax.set_xlabel('Fold angle  (°)')
    ax.set_ylabel(labels[j] + ' / panel size')
    ax.set_title(labels[j])
    ax.legend(fontsize=7)
    ax.grid(True, linewidth=0.5, alpha=0.5)

plt.suptitle('Miura-ori: deployed dimensions vs fold angle', y=1.02)
plt.tight_layout()
plt.show()
```

3.2 One Degree of Freedom

The Miura-ori's most remarkable property is that the entire tessellation has exactly one mechanical degree of freedom. Specify γ and every fold angle in the structure is determined.

This follows from the flat-foldability constraints. Each interior vertex of the Miura-ori satisfies Kawasaki's theorem with sector angles α and $\pi - \alpha$, and Maekawa's theorem with alternating M-V-M-V assignments. The constraints are so tightly coupled that choosing one fold angle propagates constraints uniquely to every other fold angle.

We can verify this numerically: build the constraint system and check its rank.

```
def miura_fold_angles(alpha, gamma):
    """
    For a Miura-ori vertex with sector angle alpha, compute the fold angle
    of each crease given the primary fold angle gamma.
    Returns all four fold angles [phi1, phi2, phi3, phi4].
    """
    # From kinematic analysis: two distinct fold angles alternate
    # phi_major = gamma (the primary fold)
    # phi_minor from Kawasaki closure at each vertex
    sa, ca = np.sin(alpha), np.cos(alpha)
    sg, cg = np.sin(gamma), np.cos(gamma)
```

```
    # Minor fold angle from closure condition (derived from Schenk & Guest)
    cos_phi_minor = (ca**2 * sg**2 - sa**2) / (ca**2 * sg**2 + sa**2)
    phi_minor = np.arccos(np.clip(cos_phi_minor, -1, 1))

    return gamma, phi_minor, gamma, phi_minor


alpha = np.radians(60)
gammas_test = np.linspace(0.05, np.pi / 2, 8)

print(f"{'  (°)':>8} {' _major (°)':>12} {' _minor (°)':>12}")
print("-" * 36)
for g in gammas_test:
    phi1, phi2, _, _ = miura_fold_angles(alpha, g)
    print(f"{np.degrees(g):8.1f} {np.degrees(phi1):12.3f} {np.degrees(phi2):12.3f}")
```

3.3 Compaction Ratio

The compaction ratio ρ is the ratio of the collapsed footprint to the deployed footprint. For the Miura-ori, this is purely a function of α:

$$\rho(\alpha) = \frac{W(\gamma = 0)}{W(\gamma = \pi/2)} = \sin\alpha \tag{3.1}$$

A smaller α gives a better compaction ratio — the structure collapses to a smaller fraction of its deployed size. But smaller α also means the crease pattern is more acute, which can create manufacturing difficulties and higher elastic strain in real materials.

```
# Compaction ratio vs alpha
alphas = np.linspace(5, 85, 300)
rho = np.sin(np.radians(alphas))

fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(alphas, rho, color='steelblue', linewidth=2)
ax.fill_between(alphas, 0, rho, alpha=0.1, color='steelblue')
ax.set_xlabel('Sector angle   (°)')
ax.set_ylabel('Compaction ratio   = sin  ')
ax.set_title('Miura-ori compaction ratio vs sector angle')
ax.set_xlim(0, 90)
ax.set_ylim(0, 1)
ax.axhline(0.5, color='coral', linestyle='--', linewidth=1, label='  = 0.5')
ax.axvline(30, color='coral', linestyle=':', linewidth=1, label='  = 30°')
ax.legend()
ax.grid(True, linewidth=0.5, alpha=0.5)
plt.tight_layout()
plt.show()
```

3.4 Optimizing α for a Target Compaction Ratio

Suppose we need a solar panel array that folds to at most 40% of its deployed width — a compaction ratio $\rho \leq 0.40$ — but we want to maximize the sector angle α (to ease manufacturing). This is a simple 1D constrained optimization:

$$\max_{\alpha} \quad \alpha \quad \text{subject to} \quad \sin\alpha \leq 0.40, \quad \alpha > 0 \tag{3.2}$$

The analytic solution is $\alpha^* = \arcsin(0.40)$, but we solve it numerically to establish the pattern we will use in more complex problems later.

```
from scipy.optimize import minimize_scalar, minimize

# Objective: maximize alpha → minimize negative alpha
# Constraint: sin(alpha) <= 0.40

rho_target = 0.40

result = minimize_scalar(
    lambda alpha: -alpha,                          # maximize alpha
    bounds=(0.01, np.pi / 2),
    method='bounded',
    options={'xatol': 1e-8}
)

# But bounded scalar minimize ignores constraints. Use minimize with constraint.
def objective(alpha):
    return -alpha[0]   # maximize alpha

constraint = {'type': 'ineq', 'fun': lambda a: rho_target - np.sin(a[0])}
bounds_opt = [(0.01, np.pi / 2 - 0.01)]

sol = minimize(objective, x0=[0.3], method='SLSQP',
               bounds=bounds_opt, constraints=constraint,
               options={'ftol': 1e-10})

alpha_opt = sol.x[0]
print(f"Target compaction ratio:     {rho_target}")
print(f"Optimal sector angle:      * = {np.degrees(alpha_opt):.4f}°")
print(f"Achieved compaction:         = {np.sin(alpha_opt):.6f}")
print(f"Analytic solution:         * = {np.degrees(np.arcsin(rho_target)):.4f}°")
```

3.5 Summary

- The Miura-ori unit cell is parameterized by sector angle α and fold angle γ.
- The structure has exactly one mechanical degree of freedom: specifying γ determines every fold angle.

- Compaction ratio $\rho = \sin\alpha$ depends only on α, not on fold angle.
- Optimizing α for a target compaction ratio is a simple constrained scalar problem — a preview of the methods in Part II.

3.6 Further Reading

Schenk and Guest (2013) derives the complete kinematics of the Miura-ori and its negative Poisson's ratio behavior. Wei et al. (2013) analyzes the geometric mechanics of general pleated origami tessellations.

3.7 Exercises

1. **Poisson's ratio.** The Miura-ori has a negative Poisson's ratio: when you pull it in the x-direction, it also expands in the y-direction. Using the width and length formulas from `miura_dimensions`, compute the effective in-plane Poisson's ratio $\nu = -(\partial L/\partial W)(W/L)$ as a function of α at $\gamma = \pi/4$. Plot $\nu(\alpha)$ for $\alpha \in (5°, 85°)$.

2. **Thickness vs compaction.** For a physical solar panel array with panel thickness t (which adds to the collapsed thickness), the actual compaction ratio changes. Modify `miura_dimensions` to include panel thickness and replot the compaction ratio vs α for $t = 0, 0.01, 0.02$ (normalized units).

3. **Two-objective optimization.** Set up a two-objective optimization problem: minimize compaction ratio and minimize peak fold strain (approximated as $\propto 1/\alpha$). Generate 50 Pareto-optimal points by sweeping over a scalarized objective $w \cdot \rho + (1 - w)/\alpha$ for $w \in [0, 1]$. Plot the Pareto front.

4. **Verify one DOF.** Write a function that takes an $m \times n$ Miura-ori with sector angle α and constructs the full system of Kawasaki constraints as a matrix equation. Compute the rank and verify that the nullspace dimension equals 1 (one DOF) for any $\alpha \neq 0°, 90°$.

5. **Deployment force.** If each crease has an elastic restoring moment $\tau = k(\phi - \phi_0)$ (a torsional spring), write an expression for the total elastic energy of an $m \times n$ Miura-ori as a function of γ. Find the fold angle γ^* at which the deployment force (energy gradient) is maximum.

Chapter 4

Circle Packing and Origami Base Design

Learning Objectives

- Understand the correspondence between origami bases and circle-packing problems
- Formulate Lang's TreeMaker algorithm as a constrained quadratic program
- Implement a circle-packing solver using `scipy.optimize`
- Design origami bases for simple tree structures

Robert Lang is a physicist turned professional origami artist, and the algorithm he published in 1996 (Lang 1996) is one of the more beautiful pieces of applied mathematics to come out of the origami world. The insight is this: the problem of designing an origami base — a folded form with the right number and proportion of flaps to become a finished model — is exactly the problem of packing circles into a square without overlap.

To see why requires a detour through the theory of origami bases.

4.1 Origami Bases and Tree Structures

An origami base is a flat-foldable configuration from which a finished model is folded. Most complex origami models — insects, human figures, animals — begin from a base that has already been folded. The base has *flaps*: points or regions that will become the legs, wings, antennae, or other appendages of the finished model.

The structure of a base can be represented as a *tree graph*: a graph with no cycles where each edge represents a flap and each node is either a joint (where flaps meet) or a terminal (the tip of a flap). The edges have lengths proportional to the desired flap lengths.

Lang's insight: for a uniaxial base (a base where all flaps fold out of a single plane), there is a one-to-one correspondence between:

1. A valid base crease pattern on a square sheet
2. A packing of circles into that square

The correspondence works as follows. Each *terminal node* of the tree corresponds to a circle. The radius of the circle is proportional to the path length from that terminal node to any other terminal node through the tree (specifically, it is the "reach" of that flap — how far it can extend while leaving room for all other flaps).

A valid base exists if and only if the corresponding circles can be packed into the square without overlap. The crease pattern is then recovered from the circle arrangement by geometric construction.

4.2 Formulating the Optimization Problem

Let the square sheet have side length 1. Let there be n terminal nodes with circles of radii $r_1, \dots, r_n$ centered at positions $(x_1, y_1), \dots, (x_n, y_n)$.

The goal is to find positions and a common scale factor s such that the scaled circles pack without overlap. Specifically:

Maximize the scale factor s (larger scale = more efficient use of the paper)

Subject to:

Non-overlap: For all pairs $i \neq j$,

$$\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \geq s(r_i + r_j) \tag{4.1}$$

Inside square: For all i,

$$sr_i \leq x_i \leq 1 - sr_i \tag{4.2}$$

$$sr_i \leq y_i \leq 1 - sr_i \tag{4.3}$$

Positivity: $s > 0$

The decision variables are $\{x_i, y_i\}_{i=1}^{n}$ and s. The problem is a nonlinear program with quadratic constraints.

```
import numpy as np
from scipy.optimize import minimize, NonlinearConstraint, Bounds
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

def unpack(z, n):
    """Unpack decision vector z into (x, y, s)."""
    x = z[:n]
    y = z[n:2*n]
    s = z[2*n]
    return x, y, s

def pack(x, y, s):
    """Pack (x, y, s) into decision vector."""
    return np.concatenate([x, y, [s]])
```

```python
def circle_pack_objective(z, n):
    """Minimize negative scale factor (= maximize s)."""
    _, _, s = unpack(z, n)
    return -s


def circle_pack_constraints(z, n, radii):
    """
    Returns a vector of constraint values, each  0 when satisfied.
    Non-overlap: dist(i,j) - s*(r_i + r_j) >= 0
    Inside: x_i - s*r_i >= 0,  1 - x_i - s*r_i >= 0  (similarly for y)
    """
    x, y, s = unpack(z, n)
    r = np.asarray(radii)
    cons = []

    # Non-overlap constraints
    for i in range(n):
        for j in range(i + 1, n):
            dist = np.sqrt((x[i] - x[j])**2 + (y[i] - y[j])**2)
            cons.append(dist - s * (r[i] + r[j]))

    # Inside square
    for i in range(n):
        cons.append(x[i] - s * r[i])          # x_i >= s*r_i
        cons.append(1 - x[i] - s * r[i])      # x_i <= 1 - s*r_i
        cons.append(y[i] - s * r[i])
        cons.append(1 - y[i] - s * r[i])

    return np.array(cons)


def solve_circle_packing(radii, n_restarts=10, seed=42):
    """
    Solve the circle-packing problem for given relative radii.
    Returns best (x, y, s, success).
    """
    rng = np.random.default_rng(seed)
    n = len(radii)
    best_s = -np.inf
    best_sol = None

    for trial in range(n_restarts):
        # Random initial positions
        x0 = rng.uniform(0.1, 0.9, n)
        y0 = rng.uniform(0.1, 0.9, n)
        s0 = 0.2
```

```
        z0 = pack(x0, y0, s0)

        # Bounds: coordinates in [0,1], s in (0, 1]
        lb = np.concatenate([np.zeros(2*n), [0.01]])
        ub = np.concatenate([np.ones(2*n),  [1.0]])

        cons = {'type': 'ineq',
                'fun': lambda z: circle_pack_constraints(z, n, radii)}

        result = minimize(
            circle_pack_objective, z0,
            args=(n,),
            method='SLSQP',
            bounds=Bounds(lb, ub),
            constraints=cons,
            options={'ftol': 1e-9, 'maxiter': 1000}
        )

        if result.success or result.fun < -best_s:
            x, y, s = unpack(result.x, n)
            if s > best_s:
                best_s = s
                best_sol = (x.copy(), y.copy(), s, result.success)

    return best_sol
```

4.3 Example: Bird Base Tree

The classic bird base underlies thousands of origami models. Its tree has four terminal nodes (two wings, head, tail) connected through a central joint. We assign relative radii based on desired flap proportions.

```
# Bird base: 4 flaps
# Tree: central node connected to 4 terminals
# Relative path lengths (radii) for equal-length flaps
radii_bird = [1.0, 1.0, 1.0, 1.0]   # all flaps equal length

x, y, s, ok = solve_circle_packing(radii_bird, n_restarts=20)

print(f"Solver converged: {ok}")
print(f"Scale factor s = {s:.4f}")
print(f"Circle centers:")
for i, (xi, yi) in enumerate(zip(x, y)):
    print(f"  Circle {i+1}: ({xi:.3f}, {yi:.3f})  radius = {s * radii_bird[i]:.3f}")


def plot_circle_packing(x, y, s, radii, title="Circle packing"):
```

```
    fig, ax = plt.subplots(figsize=(5, 5))
    ax.set_xlim(-0.05, 1.05)
    ax.set_ylim(-0.05, 1.05)
    ax.set_aspect('equal')
    ax.add_patch(mpatches.Rectangle((0, 0), 1, 1, fill=False,
                                    edgecolor='black', linewidth=2))
    colors = plt.cm.Set2(np.linspace(0, 1, len(radii)))
    for i, (xi, yi, ri) in enumerate(zip(x, y, radii)):
        circle = mpatches.Circle((xi, yi), s * ri, color=colors[i],
                                 alpha=0.5, linewidth=1.5, edgecolor='k')
        ax.add_patch(circle)
        ax.text(xi, yi, f'r={ri:.1f}', ha='center', va='center', fontsize=8)
    ax.set_title(f'{title}\ns = {s:.4f}', fontsize=10)
    ax.set_xlabel('x')
    ax.set_ylabel('y')
    ax.grid(True, linewidth=0.3, alpha=0.4)
    plt.tight_layout()
    plt.show()


plot_circle_packing(x, y, s, radii_bird, "Bird base: 4 equal flaps")
```

4.4 Unequal Flaps: Insect Base

An insect model needs long legs and shorter body parts. We assign unequal radii:

```
# Insect: 6 flaps - 3 pairs of legs + head + abdomen + 2 wings
radii_insect = [2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 1.0, 1.0]

x, y, s, ok = solve_circle_packing(radii_insect, n_restarts=30, seed=7)
print(f"Solver converged: {ok},  scale = {s:.4f}")
plot_circle_packing(x, y, s, radii_insect, "Insect base: 6 legs + 2 body parts")
```

4.5 Packing Efficiency

The packing efficiency η is the fraction of the square's area covered by circles:

$$\eta = s^2 \pi \sum_{i=1}^{n} r_i^2 \tag{4.4}$$

```
def packing_efficiency(s, radii):
    return s**2 * np.pi * sum(r**2 for r in radii)

for name, r, sol in [
    ("Bird base", radii_bird, solve_circle_packing(radii_bird, n_restarts=20)),
    ("Insect base", radii_insect, solve_circle_packing(radii_insect, n_restarts=30, seed=7)
```

```
]:
    _, _, s_sol, _ = sol
    eta = packing_efficiency(s_sol, r)
    print(f"{name:15s}  s={s_sol:.4f}   ={eta:.4f} ({100*eta:.1f}%)")
```

4.6 Summary

- An origami base corresponds to a tree graph; each terminal node maps to a circle.
- Valid base design requires packing those circles into the square sheet without overlap.
- The optimization problem maximizes the scale factor s subject to non-overlap and boundary constraints.
- The problem is a nonlinear program, solvable with `scipy.optimize.minimize` (SLSQP).
- Multiple random restarts improve solution quality for problems with many local optima.

4.7 Further Reading

Lang (1996) is the original paper — concise and readable. Lang (2011) (Chapter 12) gives a detailed treatment of the TreeMaker algorithm with worked examples.

4.8 Exercises

1. **Verify non-overlap.** After solving, write a function that checks all pairwise distances between circle centers and verifies that no two circles overlap. Report the minimum separation distance.

2. **Three-flap triangle.** Design a base for a tree with three equal-length terminal flaps. Experiment with different initial positions and count how many distinct local optima you find.

3. **Unequal radii sensitivity.** Take the bird base problem and vary one radius from 0.5 to 2.0 while holding the others fixed. Plot the optimal scale factor s^* as a function of the varied radius. What happens as one circle becomes very large?

4. **5-circle packing.** Solve the packing problem for 5 equal circles. The known optimal packing efficiency for 5 equal circles in a square is approximately $\eta \approx 0.707$. How close does your optimizer get?

5. **Gradient of the scale factor.** Compute the gradient of the objective $-s$ with respect to circle positions (x_i, y_i) analytically, and verify it matches the numerical gradient used by SLSQP. (Use `scipy.optimize.check_grad`.)

Chapter 5

Energy Minimization and Equilibrium Folds

Learning Objectives

- Model elastic fold energy as a sum of torsional spring potentials
- Formulate the equilibrium-finding problem as unconstrained and constrained minimization
- Compare gradient descent, L-BFGS-B, and SLSQP on the same problem
- Interpret convergence behavior and identify ill-conditioning

Not every fold pattern has a single, clean, analytically-determined configuration. The Miura-ori folds along a one-parameter path because its geometry enforces it. But most real structures — partially folded panels, curved crease patterns, origami with elastic materials — settle into an equilibrium that balances the external loading against the elastic energy stored in the creases.

Finding that equilibrium is an energy minimization problem.

5.1 The Elastic Fold Energy Model

Model each crease as a torsional spring: it has a natural (rest) fold angle ϕ_0 and resists deformation with stiffness k. The elastic energy stored in crease i when its fold angle is ϕ_i is:

$$E_i = \frac{k_i}{2}(\phi_i - \phi_{0,i})^2 \tag{5.1}$$

The total energy of the structure is:

$$E_{\text{total}} = \sum_{i=1}^{N} \frac{k_i}{2}(\phi_i - \phi_{0,i})^2 \tag{5.2}$$

At equilibrium, E_{total} is minimized subject to the geometric constraints from Chapter 2 — specifically, the Kawasaki closure conditions at each vertex.

This is a constrained nonlinear program:

$$\min_{\phi} \quad E_{\text{total}}(\phi) \quad \text{subject to} \quad g_j(\phi) = 0 \quad j = 1, \ldots, M \tag{5.3}$$

where the g_j are the Kawasaki constraints (or more generally, the geometric closure conditions at each vertex).

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

def total_energy(phi, phi0, k):
    """Total elastic energy of a crease pattern."""
    return 0.5 * np.sum(k * (phi - phi0)**2)

def energy_gradient(phi, phi0, k):
    """Analytic gradient of total energy."""
    return k * (phi - phi0)
```

5.2 Example: Four-Crease Vertex Under Load

Consider a single four-crease vertex (the Miura-ori unit vertex) with sector angles α and $\pi - \alpha$. The vertex has four fold angles $\phi_1, \phi_2, \phi_3, \phi_4$ subject to the Kawasaki closure condition.

We will minimize the elastic energy over the feasible set — the manifold of fold-angle vectors satisfying Kawasaki — and trace how the equilibrium configuration depends on the stiffness distribution.

```
def kawasaki_residual(phi, alpha):
    """
    Kawasaki closure condition for a 4-crease Miura vertex.
    Returns scalar: should be 0 at valid configurations.
    This is the product-of-rotations closure, linearized near flat.
    Approximation: use the alternating sum condition for small angles.
    """
    phi1, phi2, phi3, phi4 = phi
    ca = np.cos(alpha)
    sa = np.sin(alpha)
    # Exact closure from Schenk & Guest (2013): relates phi_major and phi_minor
    # cos(phi2) = (ca^2 * sin^2(phi1) - sa^2) / (ca^2 * sin^2(phi1) + sa^2)
    # Residual for phi2 given phi1:
    sg1 = np.sin(phi1)
    denom = ca**2 * sg1**2 + sa**2
    if abs(denom) < 1e-12:
        return 0.0
    cos_phi2_required = (ca**2 * sg1**2 - sa**2) / denom
    cos_phi2_required = np.clip(cos_phi2_required, -1, 1)
    phi2_required = np.arccos(cos_phi2_required)
```

```
    # Constraint: phi2 - phi2_required = 0; phi3 = phi1; phi4 = phi2
    return phi2 - phi2_required


def vertex_energy_constrained(alpha, phi0, k, phi_init=None):
    """
    Minimize elastic energy of a 4-crease vertex subject to Kawasaki closure.
    Decision variables: [phi1, phi2] - phi3=phi1, phi4=phi2 by symmetry.
    """
    if phi_init is None:
        phi_init = np.array([0.5, 0.3])

    def energy_2d(pq):
        phi1, phi2 = pq
        phi = np.array([phi1, phi2, phi1, phi2])
        return total_energy(phi, phi0, k)

    def kawasaki_2d(pq):
        phi1, phi2 = pq
        sg1 = np.sin(phi1)
        ca = np.cos(alpha)
        sa = np.sin(alpha)
        denom = ca**2 * sg1**2 + sa**2
        cos_phi2_req = (ca**2 * sg1**2 - sa**2) / (max(denom, 1e-12))
        cos_phi2_req = np.clip(cos_phi2_req, -1, 1)
        return phi2 - np.arccos(cos_phi2_req)

    result = minimize(
        energy_2d,
        phi_init,
        method='SLSQP',
        bounds=[(0.01, np.pi - 0.01)] * 2,
        constraints={'type': 'eq', 'fun': kawasaki_2d},
        options={'ftol': 1e-10, 'maxiter': 500}
    )
    phi1_eq, phi2_eq = result.x
    phi_eq = np.array([phi1_eq, phi2_eq, phi1_eq, phi2_eq])
    return phi_eq, result.fun, result.success


# Study: how does equilibrium change with rest angle phi0?
alpha = np.radians(45)
phi0_sweep = np.linspace(0.1, np.pi * 0.9, 20)
k_uniform = np.ones(4)

phi1_eq_vals, phi2_eq_vals = [], []
for phi0_val in phi0_sweep:
```

```
    phi0 = np.full(4, phi0_val)
    phi_eq, E_eq, _ = vertex_energy_constrained(alpha, phi0, k_uniform,
                                                phi_init=[phi0_val * 0.8, phi0_val * 0.5])
    phi1_eq_vals.append(np.degrees(phi_eq[0]))
    phi2_eq_vals.append(np.degrees(phi_eq[1]))

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(np.degrees(phi0_sweep), phi1_eq_vals, 'o-', color='steelblue', label='  (major fold)',
ax.plot(np.degrees(phi0_sweep), phi2_eq_vals, 's-', color='coral', label='  (minor fold)', mar
ax.plot(np.degrees(phi0_sweep), np.degrees(phi0_sweep), 'k--', linewidth=0.8, label='  (rest a
ax.set_xlabel('Rest angle   (°)')
ax.set_ylabel('Equilibrium fold angle (°)')
ax.set_title(f'Miura-ori vertex equilibrium (  = 45°, uniform stiffness)')
ax.legend()
ax.grid(True, linewidth=0.5, alpha=0.5)
plt.tight_layout()
plt.show()
```

5.3 Comparing Optimizers

The energy minimization problem Equation 5.3 is smooth and has analytic gradients. Different optimizers make different trade-offs between convergence speed, memory, and constraint-handling capability.

```
import time

def run_optimizer(method, phi0, k, alpha, phi_init):
    """
    Run one optimizer and return (solution, n_iter, wall_time, success).
    """
    history = {'phi': [], 'E': []}

    def energy_with_log(pq):
        phi = np.array([pq[0], pq[1], pq[0], pq[1]])
        E = total_energy(phi, phi0, k)
        history['phi'].append(pq.copy())
        history['E'].append(E)
        return E

    def kawasaki_2d(pq):
        phi1, phi2 = pq
        sg1 = np.sin(phi1)
        ca = np.cos(alpha)
        sa = np.sin(alpha)
        denom = ca**2 * sg1**2 + sa**2
        cos_phi2_req = np.clip((ca**2 * sg1**2 - sa**2) / max(denom, 1e-12), -1, 1)
        return phi2 - np.arccos(cos_phi2_req)
```

```
    kwargs = dict(
        fun=energy_with_log,
        x0=phi_init,
        bounds=[(0.01, np.pi - 0.01)] * 2,
        options={'maxiter': 2000}
    )

    if method == 'SLSQP':
        kwargs['method'] = 'SLSQP'
        kwargs['constraints'] = {'type': 'eq', 'fun': kawasaki_2d}
        kwargs['options']['ftol'] = 1e-10
    elif method == 'L-BFGS-B':
        kwargs['method'] = 'L-BFGS-B'
        kwargs['options']['ftol'] = 1e-12

    t0 = time.perf_counter()
    result = minimize(**kwargs)
    dt = time.perf_counter() - t0

    return result.x, history['E'], dt, result.success


alpha = np.radians(60)
phi0 = np.array([1.2, 0.6, 1.2, 0.6])
k = np.array([1.0, 2.0, 1.0, 0.5])
phi_init = np.array([0.8, 0.4])

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

for method, color in [('SLSQP', 'steelblue'), ('L-BFGS-B', 'coral')]:
    sol, E_hist, dt, ok = run_optimizer(method, phi0, k, alpha, phi_init.copy())
    axes[0].semilogy(E_hist, color=color, label=f'{method} ({len(E_hist)} evals, {dt*1000:.
    print(f"{method:10s}  converged={ok}  *={np.degrees(sol)}°  E*={E_hist[-1]:.6f}  t={dt*

axes[0].set_xlabel('Function evaluations')
axes[0].set_ylabel('Elastic energy E')
axes[0].set_title('Convergence comparison')
axes[0].legend(fontsize=8)
axes[0].grid(True, linewidth=0.5, alpha=0.5)

# Energy landscape for SLSQP solution
phi1_vals = np.linspace(0.1, np.pi - 0.1, 80)
E_landscape = []
for p1 in phi1_vals:
    sg1 = np.sin(p1)
    ca, sa = np.cos(alpha), np.sin(alpha)
    denom = ca**2 * sg1**2 + sa**2
```

```
    cos_p2 = np.clip((ca**2 * sg1**2 - sa**2) / max(denom, 1e-12), -1, 1)
    p2 = np.arccos(cos_p2)
    phi = np.array([p1, p2, p1, p2])
    E_landscape.append(total_energy(phi, phi0, k))

axes[1].plot(np.degrees(phi1_vals), E_landscape, color='steelblue', linewidth=2)
axes[1].set_xlabel('Primary fold angle   (°)')
axes[1].set_ylabel('Elastic energy E (on Kawasaki manifold)')
axes[1].set_title('Energy along the constraint manifold')
axes[1].axvline(np.degrees(sol[0]), color='coral', linestyle='--', label=f'Optimum  *={np.degr
axes[1].legend()
axes[1].grid(True, linewidth=0.5, alpha=0.5)

plt.tight_layout()
plt.show()
```

5.4 Non-Uniform Stiffness and Multiple Equilibria

Real origami structures often have creases with different stiffnesses — due to material variation, different crease depths, or deliberate design. When stiffness is non-uniform, the energy landscape can develop multiple local minima.

```
# Sweep stiffness ratio k2/k1 and track equilibrium
alpha = np.radians(45)
phi0 = np.array([1.0, 1.0, 1.0, 1.0])
k_ratios = np.logspace(-1, 1, 30)
phi1_eq_list = []

for ratio in k_ratios:
    k = np.array([1.0, ratio, 1.0, ratio])
    phi_eq, _, _ = vertex_energy_constrained(alpha, phi0, k, phi_init=[0.6, 0.5])
    phi1_eq_list.append(np.degrees(phi_eq[0]))

fig, ax = plt.subplots(figsize=(6, 4))
ax.semilogx(k_ratios, phi1_eq_list, 'o-', color='steelblue', markersize=4)
ax.axhline(np.degrees(phi0[0]), color='gray', linestyle='--', linewidth=0.8, label='Rest angle
ax.set_xlabel('Stiffness ratio k /k ')
ax.set_ylabel('Equilibrium  * (°)')
ax.set_title('Equilibrium fold angle vs stiffness ratio ( =45°)')
ax.legend()
ax.grid(True, linewidth=0.5, alpha=0.5)
plt.tight_layout()
plt.show()
```

5.5 Summary

- Elastic fold energy is a sum of torsional spring potentials: $E = \frac{1}{2}\sum k_i(\phi_i - \phi_{0,i})^2$.

- Equilibrium configurations minimize total energy subject to geometric closure constraints.
- SLSQP handles equality constraints directly; L-BFGS-B is faster for unconstrained problems.
- Non-uniform stiffness shifts the equilibrium and can introduce multiple local minima.
- The energy landscape along the Kawasaki constraint manifold is typically unimodal for a single vertex.

5.6 Further Reading

Schenk and Guest (2013) analyzes the elastic behavior of the Miura-ori under in-plane loads. Dudte et al. (2016) uses energy minimization to program curvature into origami tessellations.

5.7 Exercises

1. **Analytic gradient check.** For the 2D energy function over (φ_1, φ_2), compute the analytic gradient ∇E and verify it against `scipy.optimize.check_grad`. Report the maximum absolute error.

2. **Newton's method.** The energy has an analytic Hessian (it is constant — why?). Implement one step of Newton's method from a given starting point and verify it converges in one iteration for the unconstrained problem.

3. **Multiple restarts.** Generate 100 random starting points for the constrained vertex problem and solve from each. Plot the distribution of converged energy values. Are there multiple local minima, or does SLSQP always find the same solution?

4. **Temperature analogy.** In statistical mechanics, systems sample configurations with probability $p \propto e^{-E/T}$ at temperature T. Write a simple Metropolis sampler that samples fold angles from this distribution on the Kawasaki constraint manifold (project each MCMC step back onto the manifold). Plot the sampled fold angle distribution for $T = 0.01, 0.1, 1.0$.

5. **Stiffness design.** Find the stiffness vector $\mathbf{k}$ (with $\|\mathbf{k}\| = 1$) that places the energy minimum closest to a target fold angle $\phi^* = [\pi/3, \pi/4, \pi/3, \pi/4]$. (Hint: this is itself an optimization problem over $\mathbf{k}$.)

Chapter 6

Inverse Design

Learning Objectives

- Formulate the inverse origami design problem as a nonlinear program
- Recover Miura-ori parameters from target deployed dimensions
- Explore the Pareto front when multiple objectives conflict
- Understand when the inverse problem is ill-posed and how to regularize it

Everything up to this point has been *forward*: given a crease pattern, find the deployed shape, the compaction ratio, the equilibrium configuration. Engineers rarely work forward. They start with requirements — a solar panel that deploys to exactly 2m × 0.5m and collapses to 200mm diameter — and work backward to find the crease pattern.

This is the inverse problem. It is harder, more interesting, and usually under-determined: many crease patterns may satisfy the same set of requirements.

6.1 The Inverse Miura-ori Problem

For the Miura-ori with $m \times n$ unit cells and panel dimensions $a \times b$ (width × length), the deployed and collapsed dimensions are functions of α, γ, m, n, a, b.

From Chapter 3, in the fully deployed state ($\gamma = \pi/2$):

$$W_{\text{deployed}} = 2m \cdot a \cos\alpha \tag{6.1}$$

$$L_{\text{deployed}} = n \cdot b \tag{6.2}$$

In the fully collapsed state ($\gamma \to 0$):

$$W_{\text{collapsed}} = 2m \cdot a \sin\alpha \cos\alpha \tag{6.3}$$

The compaction ratio is $\rho = W_{\text{collapsed}}/W_{\text{deployed}} = \sin\alpha$.

Inverse problem: Given target W^*_{deployed}, L^*_{deployed}, and $\rho^* = W^*_{\text{collapsed}}/W^*_{\text{deployed}}$, find (α, m, n, a, b).

For fixed grid m, n, the continuous parameters (α, a, b) are determined by three equations:

$$2m \cdot a\cos\alpha = W^* \quad \Rightarrow \quad a = \frac{W^*}{2m\cos\alpha} \tag{6.4}$$

$$n \cdot b = L^* \quad \Rightarrow \quad b = \frac{L^*}{n} \tag{6.5}$$

$$\sin\alpha = \rho^* \tag{6.6}$$

The last equation gives $\alpha = \arcsin(\rho^*)$ directly. The system is exactly determined for fixed m, n.

```python
import numpy as np
from scipy.optimize import minimize, differential_evolution
import matplotlib.pyplot as plt

def miura_forward(alpha, m, n, a, b, gamma=np.pi/2):
    """Compute deployed and collapsed dimensions for given Miura-ori parameters."""
    sa, ca = np.sin(alpha), np.cos(alpha)
    sg, cg = np.sin(gamma), np.cos(gamma)
    W_deployed  = 2 * m * a * ca
    L_deployed  = n * b
    W_collapsed = 2 * m * a * sa * ca   # approximate (exact only at →0)
    rho = sa
    return {'W_dep': W_deployed, 'L_dep': L_deployed,
            'W_col': W_collapsed, 'rho': rho, 'alpha': alpha, 'a': a, 'b': b}


def solve_miura_inverse_fixed_grid(W_target, L_target, rho_target, m, n):
    """
    Given target deployed dimensions and compaction ratio,
    recover (alpha, a, b) analytically for fixed grid m x n.
    """
    alpha = np.arcsin(rho_target)
    a = W_target / (2 * m * np.cos(alpha))
    b = L_target / n
    result = miura_forward(alpha, m, n, a, b)
    return alpha, a, b, result


# Example: solar panel requirements
W_target  = 2.0    # m deployed width
L_target  = 0.5    # m deployed length
rho_target = 0.15  # collapse to 15% of deployed width
```

```
m, n = 6, 4   # 6 columns, 4 rows of unit cells

alpha, a, b, fwd = solve_miura_inverse_fixed_grid(W_target, L_target, rho_target, m, n)
print("=== Inverse Design Result ===")
print(f"Target:   W_dep={W_target}m  L_dep={L_target}m  rho={rho_target}")
print(f"Solution:   = {np.degrees(alpha):.3f}°  a = {a*1000:.2f}mm  b = {b*1000:.2f}mm")
print(f"Verify:   W_dep={fwd['W_dep']:.4f}m  L_dep={fwd['L_dep']:.4f}m  rho={fwd['rho']:.4f
```

6.2 When the Grid is Also a Variable

If we allow m and n to vary (they are integers), the problem becomes a mixed-integer nonlinear program. For small ranges, we can sweep all feasible grid combinations and pick the best.

```
def panel_aspect_ratio(alpha, m, n, W_target, L_target):
    """Return aspect ratio a/b for given grid and targets."""
    a = W_target / (2 * m * np.cos(alpha))
    b = L_target / n
    return a / b

# Grid sweep: find all (m,n) combinations within panel aspect ratio bounds
W_target, L_target, rho_target = 2.0, 0.5, 0.15
alpha_fixed = np.arcsin(rho_target)
ar_min, ar_max = 0.5, 2.0   # acceptable panel aspect ratios

results = []
for m in range(2, 20):
    for n in range(2, 12):
        ar = panel_aspect_ratio(alpha_fixed, m, n, W_target, L_target)
        if ar_min <= ar <= ar_max:
            a = W_target / (2 * m * np.cos(alpha_fixed))
            b = L_target / n
            results.append({'m': m, 'n': n, 'a': a*1000, 'b': b*1000,
                            'aspect': ar, 'n_panels': m * n})

results.sort(key=lambda x: abs(x['aspect'] - 1.0))   # prefer square panels

print(f"{'m':>4} {'n':>4} {'a(mm)':>8} {'b(mm)':>8} {'aspect':>8} {'panels':>7}")
print("-" * 45)
for r in results[:10]:
    print(f"{r['m']:>4} {r['n']:>4} {r['a']:>8.2f} {r['b']:>8.2f} {r['aspect']:>8.4f} {r['n
```

6.3 Multi-Objective Inverse Design

Real requirements conflict. You want: 1. Small compaction ratio ρ (good — collapses well) 2. Near-square panels (good — easier to manufacture and less buckling risk) 3. Few panels (good — lower mass, fewer hinges)

No single design satisfies all three simultaneously. The Pareto front shows the trade-off.

```
def pareto_sweep(W_target, L_target, m, n):
    """
    Sweep alpha over feasible range, compute objectives for each.
    Returns arrays of (rho, aspect_deviation, n_panels).
    """
    alphas = np.linspace(0.05, np.pi/2 - 0.05, 300)
    rhos, aspect_devs = [], []

    for alpha in alphas:
        a = W_target / (2 * m * np.cos(alpha))
        b = L_target / n
        ar = a / b
        rhos.append(np.sin(alpha))
        aspect_devs.append(abs(ar - 1.0))  # deviation from square

    return np.array(rhos), np.array(aspect_devs)


fig, ax = plt.subplots(figsize=(7, 5))
colors = plt.cm.viridis(np.linspace(0, 1, 5))

for i, (m, n) in enumerate([(4,3), (6,4), (8,5), (10,6), (12,8)]):
    rhos, asp_devs = pareto_sweep(2.0, 0.5, m, n)
    ax.scatter(rhos, asp_devs, s=4, color=colors[i], label=f'{m}×{n} = {m*n} panels', alpha=0.

ax.set_xlabel('Compaction ratio   (lower = better collapse)')
ax.set_ylabel('Panel aspect ratio deviation |a/b - 1| (lower = more square)')
ax.set_title('Pareto front: compaction vs panel squareness\n(each curve = one grid choice)')
ax.legend(fontsize=8, markerscale=3)
ax.grid(True, linewidth=0.5, alpha=0.5)
plt.tight_layout()
plt.show()
```

6.4 Numerical Inverse Design with scipy

When the forward model is more complex (non-analytic, simulated), we solve the inverse problem numerically using a nonlinear least-squares formulation.

```
from scipy.optimize import least_squares

def residuals(params, W_target, L_target, rho_target, m, n):
    """
    Residuals for inverse design: [W_error, L_error, rho_error].
    params = [alpha, log_a, log_b]  (log-parameterize to enforce positivity)
    """
    alpha = params[0]
```

```
    a = np.exp(params[1])
    b = np.exp(params[2])
    fwd = miura_forward(alpha, m, n, a, b)
    return np.array([
        (fwd['W_dep'] - W_target) / W_target,
        (fwd['L_dep'] - L_target) / L_target,
        (fwd['rho']   - rho_target) / rho_target,
    ])


# Solve numerically (should recover same result as analytic solution)
m, n = 6, 4
x0 = np.array([0.3, np.log(0.1), np.log(0.05)])  # initial guess
result = least_squares(
    residuals, x0,
    args=(W_target, L_target, rho_target, m, n),
    method='lm',
    ftol=1e-12, xtol=1e-12
)

alpha_num = result.x[0]
a_num = np.exp(result.x[1])
b_num = np.exp(result.x[2])

print("=== Numerical Inverse Design ===")
print(f"  = {np.degrees(alpha_num):.4f}°  (analytic: {np.degrees(alpha):.4f}°)")
print(f"a = {a_num*1000:.4f}mm  (analytic: {a*1000:.4f}mm)")
print(f"b = {b_num*1000:.4f}mm  (analytic: {b*1000:.4f}mm)")
print(f"Residual norm: {np.linalg.norm(result.fun):.2e}")
```

6.5 Ill-Posedness and Regularization

The inverse problem is ill-posed when there are fewer constraints than parameters, or when multiple solutions produce nearly the same forward output. A common remedy is Tikhonov regularization: add a penalty term that prefers solutions near a nominal design θ_0:

$$\min_{\theta} \quad \|\mathbf{r}(\theta)\|^2 + \lambda\|\theta - \theta_0\|^2 \tag{6.7}$$

```
def regularized_residuals(params, W_target, L_target, rho_target, m, n, theta0, lam):
    """Residuals including Tikhonov regularization."""
    physics = residuals(params, W_target, L_target, rho_target, m, n)
    reg = np.sqrt(lam) * (params - theta0)
    return np.concatenate([physics, reg])


theta0 = np.array([np.pi/4, np.log(0.15), np.log(0.06)])  # nominal design
```

```
lambdas = [0.0, 0.01, 0.1, 1.0]

print(f"{' ':>8} {'  (°)':>10} {'a (mm)':>10} {'b (mm)':>10} {'||r||':>12}")
print("-" * 55)
for lam in lambdas:
    res = least_squares(
        regularized_residuals, theta0.copy(),
        args=(W_target, L_target, rho_target, m, n, theta0, lam),
        method='lm', ftol=1e-12
    )
    a_r = np.exp(res.x[1])
    b_r = np.exp(res.x[2])
    phys_resid = residuals(res.x, W_target, L_target, rho_target, m, n)
    print(f"{lam:>8.2f} {np.degrees(res.x[0]):>10.3f} {a_r*1000:>10.3f} {b_r*1000:>10.3f} {np.
```

6.6 Summary

- The inverse design problem: given target deployed shape and compaction ratio, recover crease parameters.
- For fixed grid (m, n), the Miura-ori inverse problem has an analytic solution.
- When the grid is variable, sweep feasible (m, n) pairs and select by secondary criteria (aspect ratio, panel count).
- Multiple objectives conflict — compaction vs. panel squareness — and the Pareto front shows the trade-off space.
- Numerical formulation via least-squares enables inverse design when no analytic solution exists; Tikhonov regularization manages ill-posedness.

6.7 Further Reading

Dudte et al. (2016) solves inverse design for curved origami tessellations using gradient-based optimization. Lang (2011) (Chapter 14) discusses the degrees of freedom in base design and how constraints determine feasibility.

6.8 Exercises

1. **Sensitivity analysis.** Using the numerical inverse design setup, compute the Jacobian $\partial \mathbf{r}/\partial \theta$ at the solution using `least_squares` output (it returns `jac`). What is the condition number? What does this tell you about the inverse problem's sensitivity to measurement noise in the target dimensions?

2. **Underdetermined case.** Add a fourth parameter (e.g., vary both a and b independently while also varying α and the aspect ratio target). The system now has four unknowns and three equations. How many solutions exist? Use `differential_evolution` to find two distinct solutions.

3. **Noisy targets.** Add Gaussian noise with $\sigma = 0.01$ to W^*, L^*, ρ^* and solve the inverse problem 100 times. Plot the distribution of recovered α values. What is the standard

deviation of α for $\sigma = 0.01$?

4. **Pareto-optimal grid selection.** For the grid sweep in this chapter, implement a proper Pareto dominance filter that retains only non-dominated (m, n) pairs when considering three objectives simultaneously: ρ, aspect deviation, and total panel count. Plot the Pareto front in 3D.

5. **Curved crease inverse problem.** (Advanced) Instead of straight creases, consider a crease pattern where the crease angle α varies linearly from α_1 at one end to α_2 at the other. The deployed shape is then a curved surface. Formulate the inverse problem of finding (α_1, α_2) to hit a target curvature, and solve it numerically.

References

Demaine, Erik D, and Joseph O'Rourke. 2007. *Geometric Folding Algorithms: Linkages, Origami, Polyhedra*. Cambridge University Press.

Dudte, Levi H, Etienne Vouga, Tomohiro Tachi, and L Mahadevan. 2016. "Programming Curvature Using Origami Tessellations." *Nature Materials* 15 (5): 583–88.

Huffman, David A. 1976. "Curvature and Creases: A Primer on Paper." *IEEE Transactions on Computers* 25 (10): 1010–19.

Kawasaki, Toshikazu. 1989. "On the Relation Between Mountain-Creases and Valley-Creases of a Flat Origami." *Proceedings of the First International Meeting of Origami Science and Technology*, 229–37.

Lang, Robert J. 1996. "A Computational Algorithm for Origami Design." *Proceedings of the Twelfth Annual Symposium on Computational Geometry*, 98–105.

Lang, Robert J. 2011. *Origami Design Secrets: Mathematical Methods for an Ancient Art*. 2nd ed. CRC Press.

Miura, Koryo. 1985. "Method of Packaging and Deployment of Large Membranes in Space." *The Institute of Space and Astronautical Science Report* 618: 1–9.

Schenk, Mark, and Simon D Guest. 2013. "Geometry of Miura-Folded Metamaterials." *Proceedings of the National Academy of Sciences* 110 (9): 3276–81.

Wei, ZY, ZV Guo, L Dudte, HY Liang, and L Mahadevan. 2013. "Geometric Mechanics of Periodic Pleated Origami." *Physical Review Letters* 110 (21): 215501.

Code Reference

This appendix collects all reusable functions defined throughout the book. Every function is importable by copying it into a local module.

Chapter 2 — Geometry

```
import numpy as np

def rotation_matrix(axis, angle):
    """Rodrigues rotation about axis by angle (radians)."""
    e = np.asarray(axis, dtype=float)
    e = e / np.linalg.norm(e)
    c, s = np.cos(angle), np.sin(angle)
    skew = np.array([
        [ 0,    -e[2],  e[1]],
        [ e[2],  0,    -e[0]],
        [-e[1],  e[0],  0   ]
    ])
    return c * np.eye(3) + (1 - c) * np.outer(e, e) + s * skew

def fold_panel(panel_vertices, crease_point, crease_axis, angle):
    """Rotate panel_vertices about crease line by angle."""
    verts = np.asarray(panel_vertices, dtype=float)
    R = rotation_matrix(crease_axis, angle)
    return (R @ (verts - crease_point).T).T + crease_point

def check_kawasaki(sector_angles):
    """Check Kawasaki's theorem. Returns (satisfied, odd_sum, even_sum)."""
    alphas = np.asarray(sector_angles)
    n = len(alphas)
    if n % 2 != 0:
        return False, None, None
    odd_sum  = np.sum(alphas[0::2])
    even_sum = np.sum(alphas[1::2])
    return np.isclose(odd_sum, np.pi) and np.isclose(even_sum, np.pi), odd_sum, even_sum

def check_maekawa(fold_types):
```

```
    """Check Maekawa's theorem. Returns (satisfied, M, V)."""
    types = np.asarray(fold_types)
    M = np.sum(types ==  1)
    V = np.sum(types == -1)
    return abs(M - V) == 2, int(M), int(V)

def is_flat_foldable_vertex(sector_angles, fold_types):
    """Full flat-foldability check for one vertex."""
    kaw_ok, s1, s2 = check_kawasaki(sector_angles)
    mae_ok, M, V   = check_maekawa(fold_types)
    return {
        'kawasaki': kaw_ok, 'maekawa': mae_ok,
        'flat_foldable': kaw_ok and mae_ok,
        'odd_angle_sum': s1, 'even_angle_sum': s2,
        'mountains': M, 'valleys': V,
    }
```

Chapter 3 — Miura-ori

```
def miura_dimensions(alpha, gamma, m=4, n=4):
    """Width, length, thickness of m×n Miura-ori (panel size = 1)."""
    sa, ca = np.sin(alpha), np.cos(alpha)
    sg, cg = np.sin(gamma), np.cos(gamma)
    W = 2 * m * np.sqrt(1 - (ca * sg)**2)
    L = (n * 2 * sa * cg / np.sqrt(sa**2 + (ca*cg)**2)
         if gamma > 0 else 2 * n * sa)
    T = (2 * ca * sg * sa / np.sqrt(sa**2 + (ca*cg)**2)
         if gamma > 0 else 0)
    return W, L, T

def miura_fold_angles(alpha, gamma):
    """Return (phi_major, phi_minor, phi_major, phi_minor) for given gamma."""
    sa, ca = np.sin(alpha), np.cos(alpha)
    sg = np.sin(gamma)
    cos_phi_minor = (ca**2 * sg**2 - sa**2) / (ca**2 * sg**2 + sa**2)
    phi_minor = np.arccos(np.clip(cos_phi_minor, -1, 1))
    return gamma, phi_minor, gamma, phi_minor
```

Chapter 4 — Circle Packing

```
from scipy.optimize import minimize, Bounds

def solve_circle_packing(radii, n_restarts=10, seed=42):
    """
    Pack circles with given relative radii into unit square.
```

```
    Returns (x, y, scale, converged).
    """
    rng = np.random.default_rng(seed)
    n = len(radii)
    best_s, best_sol = -np.inf, None

    for _ in range(n_restarts):
        x0 = np.concatenate([rng.uniform(0.1, 0.9, n),
                             rng.uniform(0.1, 0.9, n), [0.2]])
        lb = np.concatenate([np.zeros(2*n), [0.01]])
        ub = np.concatenate([np.ones(2*n),  [1.0]])

        def obj(z): return -z[2*n]

        def cons(z):
            x, y, s = z[:n], z[n:2*n], z[2*n]
            r = np.asarray(radii)
            c = []
            for i in range(n):
                for j in range(i+1, n):
                    c.append(np.hypot(x[i]-x[j], y[i]-y[j]) - s*(r[i]+r[j]))
            for i in range(n):
                c += [x[i]-s*r[i], 1-x[i]-s*r[i],
                      y[i]-s*r[i], 1-y[i]-s*r[i]]
            return np.array(c)

        res = minimize(obj, x0, method='SLSQP',
                       bounds=Bounds(lb, ub),
                       constraints={'type': 'ineq', 'fun': cons},
                       options={'ftol': 1e-9, 'maxiter': 1000})
        s_val = res.x[2*n]
        if s_val > best_s:
            best_s = s_val
            best_sol = (res.x[:n].copy(), res.x[n:2*n].copy(),
                        s_val, res.success)

    return best_sol
```

Chapter 5 — Energy Minimization

```
def total_energy(phi, phi0, k):
    """Total elastic energy of a crease pattern."""
    return 0.5 * np.sum(k * (phi - phi0)**2)

def energy_gradient(phi, phi0, k):
    """Analytic gradient of total energy."""
```

```
    return k * (phi - phi0)
```

Chapter 6 — Inverse Design

```
def miura_forward(alpha, m, n, a, b, gamma=np.pi/2):
    """Forward model: compute Miura-ori deployed/collapsed dimensions."""
    sa, ca = np.sin(alpha), np.cos(alpha)
    return {
        'W_dep':  2*m*a*ca,
        'L_dep':  n*b,
        'W_col':  2*m*a*sa*ca,
        'rho':    sa,
        'alpha':  alpha, 'a': a, 'b': b
    }

def solve_miura_inverse_fixed_grid(W_target, L_target, rho_target, m, n):
    """Analytic inverse design for fixed grid."""
    alpha = np.arcsin(rho_target)
    a = W_target / (2*m*np.cos(alpha))
    b = L_target / n
    return alpha, a, b, miura_forward(alpha, m, n, a, b)
```

Environment Setup

```
pixi install
pixi run kernel     # register Jupyter kernel
pixi run render     # render full ebook to docs/
pixi run preview    # live preview in browser
```

Dependencies: `numpy`, `scipy`, `matplotlib` — all from conda-forge.